Daytona Sandbox Compute For Agent Workloads

August 2026

Daytona runs coding agents, evaluations, and other AI workloads inside isolated sandboxes. These environments handle the work around the agent, including repository search, file edits, test execution, dependency installation, local data processing, and artifact generation.

For agent workloads, sandbox performance directly affects how quickly and reliably the agent can work. Poor performance shows up as slower task completion, unresponsive tools, higher tail latency, and in training or rollout workloads, accelerators sitting idle while they wait on the environment. As usage scales, CPU performance also determines how many concurrent environments a machine can support without degrading responsiveness.

We recently benchmarked customer-shaped workloads across NVIDIA Vera, AMD EPYC Turin, and AWS Graviton5. The results changed how we think about CPUs for agent infrastructure and what we want from future chips.

What We Learned From Vera

The clearest result was on coding-agent workloads.

A representative agent episode took roughly **7 seconds on Vera versus 17 seconds on Zen 5** at low concurrency. At millions of rollouts, that difference turns into thousands of worker-hours and can materially change accelerator utilization.

The more important result, however, was what happened under load.

Vera continued to complete agent jobs efficiently as we increased the number of sandboxes on a node. Per-job latency stayed relatively flat across agent, filesystem, and analytics workloads.

Other CPUs behaved differently:

- **Zen 5** was competitive on short eval jobs and strongest on media transcoding, but long agent workloads plateaued earlier as concurrency increased.
- **Graviton5** could look competitive at low utilization, but per-job latency increased substantially as the node became packed.
- **Vera** combined strong single-job performance with the best density characteristics for our agent workloads.

This matters because Daytona does not operate CPUs one benchmark process at a time. A production node may run hundreds of isolated 1-vCPU sandboxes or many more fractional workloads.

For us, **performance under tenancy matters as much as peak core performance.**

Why Agent Workloads Stress CPUs Differently

Coding agents are an unusual systems workload.

They spend their time moving between many small operations:

- Python execution
- repository search
- parsing and modifying source code

- pytest and other test runners
- package installation
- small-file I/O
- local databases and analytics
- compression and artifact processing

This is very different from a sustained GEMM, compiler benchmark, or large sequential workload.

The CPU repeatedly moves between compute, memory, filesystem, kernel, and process activity. At high tenancy, hundreds of sandboxes create contention across caches, memory bandwidth, storage, and other shared resources.

That creates two performance questions:

1. **How quickly can one sandbox finish an agent step?**
2. **How many sandboxes can the machine run before each one starts getting slower?**

Traditional CPU benchmarks often answer the first question only indirectly and say very little about the second.

For Daytona, both determine the economics of the machine.

What This Means For Future Chips

Our ideal CPU for agent infrastructure needs four characteristics:

- Strong burst performance on Python, tests, and filesystem-heavy workloads
- High completed-job throughput when the machine is densely packed
- Flat per-job latency as concurrency increases
- Low failure and tail-latency rates at high tenancy

Core count alone is not enough.

Our results suggest that **memory bandwidth per core, cache behavior, uncore design, and shared I/O behavior become increasingly important as sandbox density rises.**

Graviton5 was particularly useful here. It showed that ARM64 itself does not explain Vera's performance. Two ARM64 platforms can behave very differently once hundreds of isolated workloads compete for shared resources.

Likewise, Zen 5 showed an important tradeoff. It remained the strongest platform for our media workloads. We do not want to optimize a future system so heavily for Python and filesystem activity that it gives away performance on vectorized workloads such as transcoding.

The ideal future platform combines:

Vera-like agent latency and density with stronger performance across vector and media workloads.

How Daytona Can Work With Qualcomm

Daytona can provide something that conventional benchmark suites cannot: **production-shaped traces from large-scale AI agent infrastructure.**

We can evaluate Qualcomm hardware using the same workloads that run across our fleet:

- full coding-agent episodes
- short evaluation trials
- filesystem-heavy sandbox workloads
- local analytics
- package installation
- media transformation
- high-density sandbox packing

More importantly, we can characterize where performance starts to degrade as tenancy increases.

That can help identify whether a workload is limited by:

- individual core performance
- cache contention
- memory bandwidth
- memory latency
- storage
- scheduler behavior
- shared system resources

Instead of optimizing against a synthetic benchmark, we can give Qualcomm feedback from workloads that may eventually represent a significant portion of AI infrastructure CPU demand.

What Qualcomm Can Teach Us

There is also a large amount we want to learn from the hardware side.

Daytona sees performance from the sandbox and fleet perspective. Qualcomm understands what is happening deeper in the system.

Working together could help us answer questions such as:

- Which hardware counters best predict when sandbox density is becoming inefficient?
- What resources are actually saturating first on packed agent nodes?
- How should we place or schedule sandboxes around cache, NUMA, and memory topology?
- Which workload characteristics should influence CPU architecture for future agent infrastructure?
- Can the runtime expose enough information for Daytona to schedule workloads differently before contention becomes visible to customers?

That feedback can improve Daytona as well.

A better understanding of the processor lets us build a scheduler that is aware of the machine underneath it rather than treating every vCPU as interchangeable.

What We Want To Test Next

For future chips, we want to understand how well the CPU supports the environment around an AI agent.

The important workloads are not just traditional CPU benchmarks. We care about repository search, file edits, dependency installation, test execution, local data processing, and artifact generation. We also care about what happens when hundreds of those workloads share the same machine.

For each new platform, the questions are straightforward:

- How fast does a single agent workload run?
- How much throughput can the system sustain as sandbox count increases?
- How quickly does per-sandbox latency degrade under contention?
- Which shared resources become bottlenecks first?
- How predictable is performance at high density?

The Vera results showed us that these characteristics can differ substantially between CPUs, even when core counts or architectures look similar on paper.

Going forward, this is the bar we want to use when evaluating new hardware. The best CPU for Daytona is one that keeps individual agent operations fast while continuing to perform well as the machine fills with sandboxes.

Ultimately, we want the agent environment to become a high-performance execution layer that keeps every tool call responsive, supports massive concurrency, and delivers predictable performance so compute, memory, storage, and I/O never become hidden constraints on what agents can accomplish.